

Knapsack

Day 3 solution walkthrough

Competitive Programming Bootcamp

Statement excerpts:

- ▶ *“greatest value that fit into the knapsack”*
- ▶ *“indices of the chosen items”*

Programming interpretation:

- ▶ This is 0/1 knapsack: each item can be taken at most once.
- ▶ The score is value, while the constraint is total weight at most capacity.
- ▶ Because output requires item indices, the DP must support reconstruction.

Textbook connection: CP4 3.5 c. Knapsack / Subset-Sum

Core technique: 0/1 knapsack DP with reconstruction

- ▶ Each item has a value and weight.
- ▶ The capacity limits total selected weight.
- ▶ The output asks for selected item indices, not just the best value.

- ▶ Each item can be taken at most once, so this is 0/1 knapsack.
- ▶ A one-dimensional dp array gives best value per capacity.
- ▶ A chosen table is needed to reconstruct the selected indices.

- ▶ Initialize $dp[w] = 0$ for every capacity w .
- ▶ For each item i , scan capacities backward from capacity to weight.
- ▶ If $dp[w - \text{weight}] + \text{value}$ improves $dp[w]$, update $dp[w]$.
- ▶ Mark $\text{chosen}[i][w] = 1$ for improved states.
- ▶ Reconstruct by walking items backward and subtracting chosen item weights.

- ▶ Backward capacity order prevents using item i more than once.
- ▶ $\text{chosen}[i][w]$ means item i was responsible for the best known value at capacity w .
- ▶ The reconstruction follows those choices from the final capacity.

- ▶ After processing i items, $dp[w]$ is the best value using only those items within capacity w .
- ▶ The transition considers both excluding and including the current item.
- ▶ The chosen table records exactly the include decisions that produced the final DP value.

Complexity: $O(NC)$ time and $O(NC)$ memory because reconstruction stores choices for every item/capacity.

- ▶ Multiple test cases are read until EOF.
- ▶ If only the value were required, memory could be reduced further.

Source lines: knapsack.py:33-42

```
import sys

def solve(capacity, n, items):
    dp = [0] * (capacity + 1)

    chosen = [bytearray(capacity + 1) for _ in range(n)]
```

What this chunk does

- ▶ `dp[w]` stores the best value available with capacity `w`.
- ▶ `chosen[i][w]` remembers that item `i` improved capacity `w`.
- ▶ The bytearray table keeps reconstruction memory compact.

Source lines: knapsack.py:44-53

```
for i in range(n):  
    value, weight = items[i]  
  
    for w in range(capacity, weight - 1, -1):  
        new_value = dp[w - weight] + value  
  
        if new_value > dp[w]:  
            dp[w] = new_value  
            chosen[i][w] = 1
```

What this chunk does

- ▶ For each item, capacities are scanned backward.
- ▶ Backward scanning prevents reusing the same item multiple times.
- ▶ When taking the item improves value, both dp and chosen are updated.

Source lines: knapsack.py:55-66

```
result = []
remaining = capacity

for i in range(n - 1, -1, -1):
    if chosen[i][remaining]:
        result.append(i)
        remaining -= items[i][1]

result.reverse()

return result
```

What this chunk does

- ▶ Reconstruction starts at full capacity after all items are processed.
- ▶ If `chosen[i][remaining]` is set, item `i` was used in the optimal path.
- ▶ The result is reversed because reconstruction walks from last item to first.

Source lines: knapsack.py:69-89

```
def main():
    input = sys.stdin.buffer.readline
    output = []

    while True:
        line = input()

        if not line:
            break

        if not line.strip():
            continue

        capacity, n = map(int, line.split())

        items = []

        for _ in range(n):
            value, weight = map(int, input().split())
            items.append((value, weight))
```

What this chunk does

- ▶ The input has an unknown number of test cases, so the loop stops at EOF.
- ▶ Blank lines are skipped defensively.
- ▶ Each case reads the capacity and all value/weight pairs.

Source lines: knapsack.py:91-102

```
result = solve(capacity, n, items)

output.append(str(len(result)))

output.append(" ".join(map(str, result)))

sys.stdout.write("\n".join(output))

main()
```

What this chunk does

- ▶ solve returns the chosen item indices for the case.
- ▶ The final output follows Kattis: count first, then selected indices.
- ▶ All output lines are buffered and written once.

Solution file: `practice_contests/day_3/knapsack.py`

Problem: <https://open.kattis.com/problems/knapsack>

Use this deck with the source code open beside it. The slides explain the reasoning; the code shows the exact input handling and output formatting.